UNIT I

Python Basics & Control Flow

History · Anaconda & Jupyter · Identifiers, variables, operators, data types · Input/Output · Type conversion · if / loops / try-except · Sample Programs 1–6

➤ What you will learn in this unit

- Where Python came from, what it is used for, and how to set up Anaconda and Jupyter Notebook.
- Building blocks: identifiers, keywords, variables, operators, data types, indentation, comments, input and output.
- Controlling a program with **if** statements, **while** / **for** loops, **break**, **continue** and **try–except**.
- Solutions of the sample programs **1 to 6** with real output.

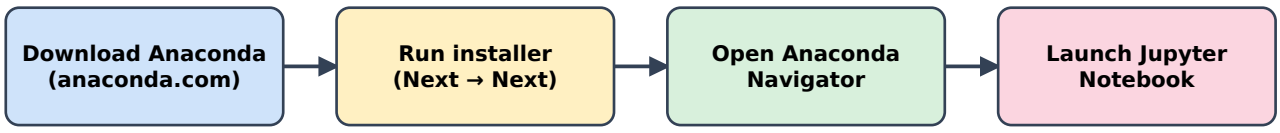
1. History and thrust areas of Python

- Created by **Guido van Rossum** at CWI, Netherlands; development began in **1989**, first release (0.9) in **1991**.
- Named after the comedy show *"Monty Python's Flying Circus"* — not after the snake.
- Python 2.0 (2000) → **Python 3.0 (2008)**. Python 2 is now retired; always use Python 3.
- Features: simple & readable, **interpreted**, free & open source, **dynamically typed**, object-oriented, portable, huge standard library.

Thrust (application) area	Examples / libraries
Data Science & Analytics	NumPy, Pandas, Matplotlib
Artificial Intelligence & Machine Learning	scikit-learn, TensorFlow, PyTorch
Web development	Django, Flask
Automation & scripting	Automating files, emails, testing
Desktop / games / IoT	Tkinter, Pygame, Raspberry Pi

2. Installing Anaconda and using Jupyter Notebook

Anaconda is a free Python distribution that installs Python, the **conda** package manager and popular data-science libraries (NumPy, Pandas, Matplotlib, Jupyter…) in one go.



Jupyter opens in your web browser — write code in cells, press Shift + Enter to run

- A Jupyter notebook (**.ipynb**) is made of **cells**: *Code* cells run Python, *Markdown* cells hold notes/headings.
- Output appears directly below the cell, so it is ideal for learning, data science and sharing results.

- Common shortcuts: **Shift+Enter** run cell · **A** / **B** insert cell above / below · **DD** delete cell · **M** Markdown · **Y** Code.

3. Parts of the Python language

3.1 Identifiers and keywords

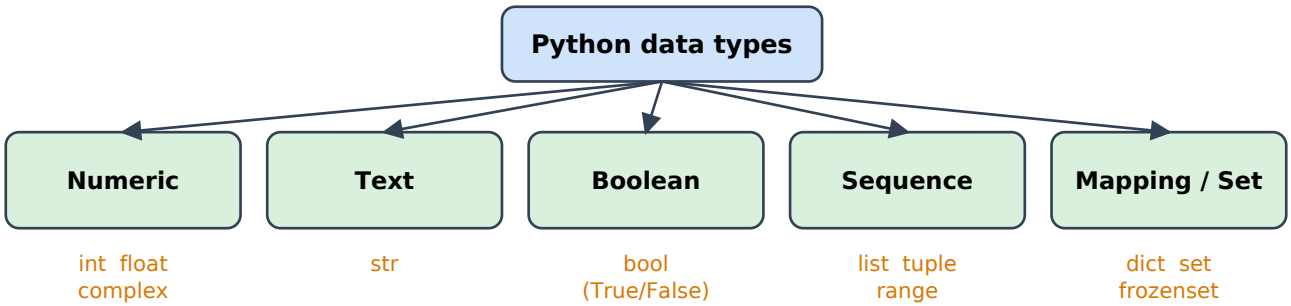
An **identifier** is the name given to a variable, function, class etc. Rules: it may contain letters, digits and underscore; it **cannot start with a digit**; it is **case-sensitive** (*Age* ≠ *age*); it cannot be a keyword and cannot contain spaces or symbols.

Keywords are reserved words with a special meaning, e.g. `if else elif while for break continue def class return True False None and or not in is try except`. Print them with `import keyword; print(keyword.kwlist)`.

3.2 Statements, expressions, variables

- **Expression** — produces a value: `3 + 4 * 2`. **Statement** — an instruction Python executes: `x = 10`, `print(x)`.
- **Variable** — a name that refers to a value in memory. No declaration needed: `x = 5` creates it; `x, y = 1, 2` assigns many.
- **Comments** — ignored by Python: `# single line` and `"""multi-line / docstring"""`.
- **Indentation** — Python uses **spaces (4 recommended)** to mark blocks. Wrong indentation gives *IndentationError*.

3.3 Data types



Type	Example	Mutable?	Note
int	`25, -3`	No	Whole numbers of any size
float	`3.14, 2e3`	No	Decimal numbers
complex	`2+3j`	No	Real + imaginary part (j)
str	`"Python"`	No	Sequence of characters
bool	`True, False`	No	Subclass of int (True = 1)
list	`[1, 2, 3]`	**Yes**	Ordered collection
tuple	`(1, 2, 3)`	No	Ordered, fixed
set	`{1, 2, 3}`	**Yes**	Unordered, unique items
dict	`{"a": 1}`	**Yes**	Key : value pairs

3.4 Operators, precedence and associativity

Type	Operators	Example → result
Arithmetic	`+ - * / // % **`	`7 // 2` → 3 , `7 % 2` → 1 , `2 ** 3` → 8
Relational	`== != > < >= <=`	`5 > 3` → True
Assignment	`= += -= *= /= //= %= **=`	`x += 2` means `x = x + 2`
Logical	`and or not`	`True and False` → False
Bitwise	`& | ^ ~ << >>`	`5 & 3` → 1 , `5 << 1` → 10
Ternary	`a if condition else b`	`"Pass" if m>=40 else "Fail"`
Membership	`in not in`	`"a" in "cat"` → True
Identity	`is is not`	Checks whether two names refer to the **same object**

Precedence (high → low): **() → ** → +x -x ~x → * / // % → + - → << >> → & → ^ → | → comparisons, in, is → not → and → or → assignment**.

- **Associativity** decides the order when operators have equal precedence. Most operators work **left to right**; the exponent **** and assignment work **right to left**.

```
print(2 + 3 * 4)          # * before +  -> 14
print((2 + 3) * 4)        # brackets first -> 20
print(2 ** 3 ** 2)        # right to left: 2 ** 9 -> 512
print(10 - 4 - 3)         # left to right: (10-4)-3 -> 3
print(not True or True)   # not first -> True
```

Output

```
14
20
512
3
True
```

3.5 Reading input and printing output

- **input(prompt)** always returns a **string**; convert it when you need numbers.
- **print()** shows output. Useful options: *sep* (separator), *end* (what to print at the end, default newline).

```
# name = input("Enter name: ")          # keyboard input (returns str)
# age = int(input("Enter age: "))       # convert to int
name, age = "Asha", 18                   # fixed values so this demo can run
print("Name:", name, "Age:", age)
print("a", "b", "c", sep="-")             # custom separator
print("Hello", end=" ")                   # no newline at end
print("World")
print(f"{name} will be {age + 1} next year")   # f-string
```

Output

```
Name: Asha Age: 18
a-b-c
Hello World
Asha will be 19 next year
```

3.6 Type conversion, type() and the is operator

Converting one data type to another is **type conversion** (casting). Functions: `int()` `float()` `str()` `bool()` `list()` `tuple()` `complex()`. Python also converts automatically (**implicit**) — e.g. int + float gives float.

```
a = "25"
b = int(a) + 5                # explicit conversion str -> int
print(b, type(b))
print(3 + 2.5, type(3 + 2.5))  # implicit conversion -> float
print(float(7), str(3.5), bool(0), bool("hi"), int(9.99))
x = [1, 2]; y = [1, 2]; z = x
print(x == y, x is y, x is z)  # == compares values, is compares identity
```

Output

```
30 <class 'int'>
5.5 <class 'float'>
7.0 3.5 False True 9
True False True
```

3.7 Dynamic and strongly typed language

- **Dynamically typed** — you do not declare a type; the type belongs to the **value** and is decided at run time, so the same variable can hold different types at different times.
- **Strongly typed** — Python does **not** silently mix incompatible types: `"5" + 2` raises *TypeError*; you must convert explicitly.

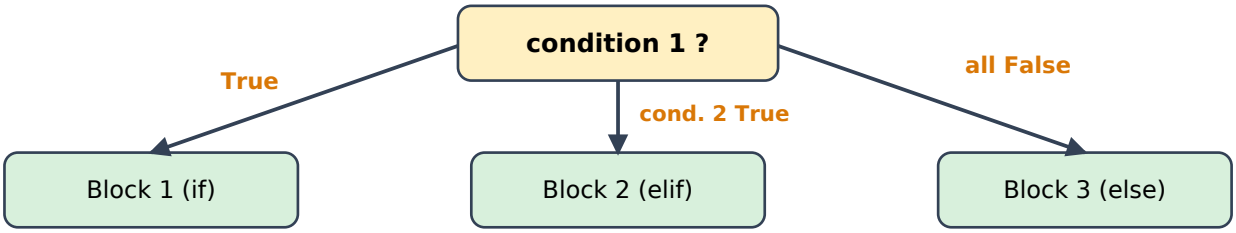
```
x = 10
print(type(x))
x = "ten"                     # same name, new type (dynamic typing)
print(type(x))
try:
    print("5" + 2)            # strong typing: no silent mixing
except TypeError as e:
    print("TypeError:", e)
```

Output

```
<class 'int'>
<class 'str'>
TypeError: can only concatenate str (not "int") to str
```

4. Control Flow Statements

4.1 Decision making: if, if-else, if…elif…else, nested if



Only ONE block runs. Python checks conditions from top to bottom.

```
marks = 72
if marks >= 90:
    print("Grade A")
```

```
elif marks >= 60:                 # checked only if the first is False
    print("Grade B")
else:
    print("Grade C")

n = 12
if n > 0:                          # nested if
    if n % 2 == 0:
        print("Positive even")
    else:
        print("Positive odd")
```

Output

```
Grade B
Positive even
```

4.2 Loops: while and for

	while loop	for loop
Use when	Number of repetitions is **not known**; repeat while condition is True	Iterating over a **sequence** / known range
Syntax	`while condition:`	`for item in sequence:`
Danger	Forgetting to update the variable → infinite loop	None — ends when the sequence ends

range(start, stop, step) gives numbers from *start* up to (but not including) *stop*: `range(5)` → 0..4, `range(1, 10, 3)` → 1, 4, 7.

```
i = 1
while i <= 3:                      # while loop
    print("while:", i)
    i += 1

for n in range(1, 10, 3):          # for loop with range
    print("for:", n)

for ch in "Hi":                    # looping over a string
    print(ch)
for i in range(2):                 # loop with else: runs when no break happens
    pass
else:
    print("loop finished normally")
```

Output

```
while: 1
while: 2
while: 3
for: 1
for: 4
for: 7
H
i
loop finished normally
```

4.3 break and continue

Statement	Effect
`break`	**Stops** the loop completely and jumps to the statement after the loop
`continue`	**Skips** the rest of the current iteration and goes to the next one
`pass`	Does nothing — a placeholder where a statement is required

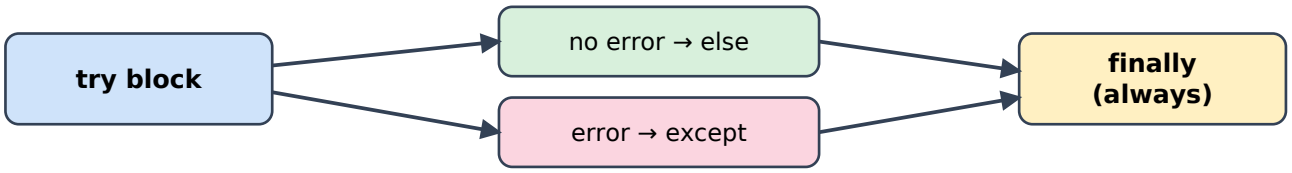
```
for i in range(1, 8):
    if i == 3:
        continue                    # skip 3
    if i == 6:
        break                       # stop at 6
    print(i, end=" ")
print()
```

Output

```
1 2 4 5
```

4.4 Catching exceptions with try and except

An **exception** is an error that occurs while the program runs (e.g. dividing by zero). Without handling, the program crashes. Put risky code in **try**; the **except** block runs only if an error occurs. **else** runs if no error; **finally** always runs.




```
try:
    a = int("abc")                  # raises ValueError
except ValueError:
    print("Please enter a valid number")
try:
    print(10 / 0)
except ZeroDivisionError as e:
    print("Error:", e)
else:
    print("no error")
finally:
    print("done")
```

Output

```
Please enter a valid number
Error: division by zero
done
```

★ Study Tip

- Remember: **Indentation = block**. A colon **:** must follow if / elif / else / while / for / def / try / except.
- For loops: **for** when you know how many times, **while** when you stop on a condition.

✓ Quick Revision

- Python: created by Guido van Rossum (1991), interpreted, dynamically & strongly typed, open source.
- Identifier rules: letters/digits/_ , cannot start with a digit, case-sensitive, not a keyword.
- Types: int, float, complex, str, bool, list, tuple, set, dict. **type()** shows the type; **is** compares identity, **==** compares value.
- Operators: arithmetic, relational, assignment, logical, bitwise, ternary, membership, identity. ** is right-associative.
- input() gives str → convert with int()/float(). print(sep=, end=).
- Control flow: if / elif / else, while, for with range(), break, continue, pass, try / except / else / finally.

SAMPLE PROGRAMS 1 – 6 (with output)

Program 1

Question

- Write a program to find the largest element among three numbers.

Program

```
# For keyboard input use: a = int(input("Enter first number: ")) etc.
a, b, c = 25, 78, 46

if a >= b and a >= c:
    largest = a
elif b >= a and b >= c:
    largest = b
else:
    largest = c

print("Numbers :", a, b, c)
print("Largest :", largest)
print("Using max():", max(a, b, c))
```

Output

```
Numbers : 25 78 46
Largest : 78
Using max(): 78
```

Program 2

Question

- Write a Program to display all prime numbers within an interval.

Program

```
start, end = 10, 50          # interval (can be read with input())

print("Prime numbers between", start, "and", end, "are:")
for num in range(start, end + 1):
    if num > 1:
        for i in range(2, int(num ** 0.5) + 1):   # check divisors up to sqrt(num)
            if num % i == 0:
                break                              # divisor found -> not prime
        else:
            print(num, end=" ")                    # loop ended without break -> prime
print()
```

Output

```
Prime numbers between 10 and 50 are:
11 13 17 19 23 29 31 37 41 43 47
```

i Remember

- A number is prime if it has no divisor between 2 and its square root. The **for…else** executes *else* only when the loop was not stopped by *break*.

Program 3

Question

- Write a program to swap two numbers without using a temporary variable.

Program

```
a, b = 10, 20
print("Before swap: a =", a, " b =", b)

# Method 1: tuple (multiple) assignment - the Pythonic way
a, b = b, a
print("After swap (tuple assignment): a =", a, " b =", b)

# Method 2: using arithmetic operators
a = a + b
b = a - b
a = a - b
print("After swap again (arithmetic): a =", a, " b =", b)

# Method 3: using XOR (bitwise)
a = a ^ b
b = a ^ b
a = a ^ b
print("After swap again (XOR)       : a =", a, " b =", b)
```

Output

```
Before swap: a = 10  b = 20
After swap (tuple assignment): a = 20  b = 10
After swap again (arithmetic): a = 10  b = 20
After swap again (XOR)       : a = 20  b = 10
```

Program 4

Question

- Demonstrate the following Operators in Python with suitable examples: i) Arithmetic ii) Relational iii) Assignment iv) Logical v) Bit wise vi) Ternary vii) Membership viii) Identity Operators.

Program

```
a, b = 17, 5

print("i) Arithmetic")
print("a + b =", a + b, "| a - b =", a - b, "| a * b =", a * b)
print("a / b =", a / b, "| a // b =", a // b, "| a % b =", a % b, "| a ** 2 =", a ** 2)

print("\nii) Relational")
print("a == b:", a == b, "| a != b:", a != b, "| a > b:", a > b, "| a <= b:", a <= b)

print("\niii) Assignment")
x = 10
x += 5;  print("x += 5  ->", x)
x -= 3;  print("x -= 3  ->", x)
x *= 2;  print("x *= 2  ->", x)
x //= 5; print("x //= 5 ->", x)
x **= 2; print("x **= 2 ->", x)

print("\niv) Logical")
print("a > 10 and b > 10:", a > 10 and b > 10)
print("a > 10 or  b > 10:", a > 10 or b > 10)
```

```
print("not (a > 10)     :", not (a > 10))

print("\nv) Bitwise  (a = 12 = 1100, b = 10 = 1010)")
p, q = 12, 10
print("p & q =", p & q, "| p | q =", p | q, "| p ^ q =", p ^ q)
print("~p =", ~p, "| p << 1 =", p << 1, "| p >> 1 =", p >> 1)

print("\nvi) Ternary")
marks = 65
print("Result:", "Pass" if marks >= 40 else "Fail")

print("\nvii) Membership")
fruits = ["apple", "mango"]
print("'apple' in fruits    :", "apple" in fruits)
print("'grape' not in fruits:", "grape" not in fruits)
print("'th' in 'python'     :", "th" in "python")

print("\nviii) Identity")
l1 = [1, 2]; l2 = [1, 2]; l3 = l1
print("l1 is l2:", l1 is l2, "| l1 is l3:", l1 is l3, "| l1 == l2:", l1 == l2)
print("l1 is not l2:", l1 is not l2)
```

Output

```
i) Arithmetic
a + b = 22 | a - b = 12 | a * b = 85
a / b = 3.4 | a // b = 3 | a % b = 2 | a ** 2 = 289

ii) Relational
a == b: False | a != b: True | a > b: True | a <= b: False

iii) Assignment
x += 5  -> 15
x -= 3  -> 12
x *= 2  -> 24
x //= 5 -> 4
x **= 2 -> 16

iv) Logical
a > 10 and b > 10: False
a > 10 or  b > 10: True
not (a > 10)     : False

v) Bitwise  (a = 12 = 1100, b = 10 = 1010)
p & q = 8 | p | q = 14 | p ^ q = 6
~p = -13 | p << 1 = 24 | p >> 1 = 6

vi) Ternary
Result: Pass

vii) Membership
'apple' in fruits    : True
'grape' not in fruits: True
'th' in 'python'     : True

viii) Identity
l1 is l2: False | l1 is l3: True | l1 == l2: True
l1 is not l2: True
```

Program 5

Question

- Write a program to add and multiply complex numbers.

Program

```
c1 = complex(3, 4)          # 3 + 4j
c2 = 2 + 5j                 # direct form

print("First number :", c1)
print("Second number:", c2)
print("Sum          :", c1 + c2)
print("Product      :", c1 * c2)
print("Real part of product:", (c1 * c2).real)
print("Imag part of product:", (c1 * c2).imag)
print("Magnitude of c1     :", abs(c1))
```

Output

```
First number : (3+4j)
Second number: (2+5j)
Sum          : (5+9j)
Product      : (-14+23j)
Real part of product: -14.0
Imag part of product: 23.0
Magnitude of c1     : 5.0
```

i Remember

- Product: (3+4j)(2+5j) = 6 + 15j + 8j + 20j² = 6 + 23j − 20 = **−14 + 23j** since j² = −1.

Program 6

Question

- Write a program to print multiplication table of a given number.

Program

```
n = 7                                  # or: n = int(input("Enter a number: "))
print("Multiplication table of", n)
for i in range(1, 11):
    print(n, "x", i, "=", n * i)
```

Output

```
Multiplication table of 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Possible Exam Questions

Short answer (2–3 marks)	Long answer (5–10 marks)
1. Who developed Python? Mention any four features. 2. What are identifiers? State the naming rules. 3. Differentiate == and *is*. 4. What is dynamic typing? 5. Differentiate break and continue. 6. What is an exception? 7. Write the output of *print(2 ** 3 ** 2)*.	1. Explain the history and thrust areas of Python. 2. Explain the steps to install Anaconda and use Jupyter Notebook. 3. Explain the different data types with examples. 4. Explain all types of operators with precedence and associativity. 5. Explain if, if-else, elif and nested if with examples. 6. Explain while and for loops with break and continue. 7. Write programs 1–6 from the sample experiments.